



UNICAMP



Aula 2: Da Especificação ao Baseline

Engenharia incremental de sistemas multiagentes

Marcelo S. Reis [†]

[†] Departamento de Sistemas de Informação
Instituto de Computação, Universidade Estadual de Campinas

Campinas, 27 de agosto de 2026

Sumário

Visão geral e estudo de caso

Especificação

CrITÉrios de sucesso e avaliação

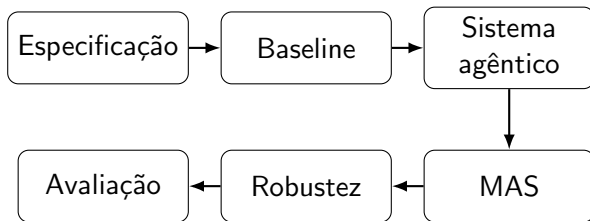
Baseline

Do baseline ao Entregável 1

Objetivos da aula

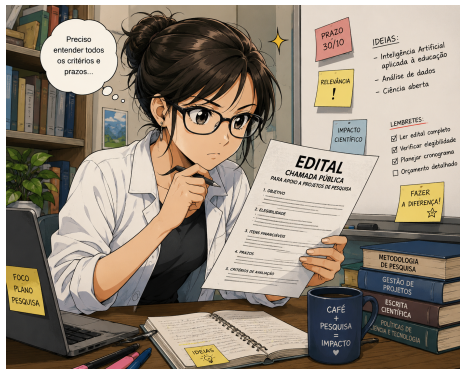
- ❖ Transformar um problema em uma especificação inicial
- ❖ Distinguir requisitos funcionais e não funcionais
- ❖ Escrever requisitos verificáveis e definir o escopo
- ❖ Definir critérios preliminares de sucesso e como medi-los
- ❖ Construir um baseline deliberadamente simples
- ❖ Identificar limitações que justifiquem as próximas arquiteturas

Trajectoria do projeto



Cada incremento deve responder a uma limitação observada na versão anterior

Estudo de caso: Análise de Editais

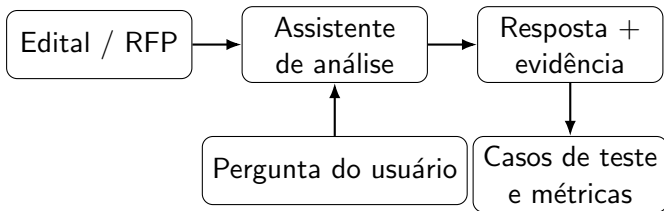


Dado um edital, chamada ou RFP (*Request For Proposal*), apoiar o usuário na identificação de:

- ❖ objetivo e escopo
- ❖ elegibilidade e requisitos
- ❖ prazos
- ❖ documentos obrigatórios
- ❖ ambiguidades e informações ausentes

Este é o exemplo que usaremos hoje; cada grupo aplicará o mesmo roteiro ao seu próprio tema

Visão do estudo de caso



O mesmo problema será mantido enquanto a arquitetura evolui

Por que este problema funciona bem?

- ❖ O baseline pode ser uma única chamada ao LLM
- ❖ Ferramentas e fontes externas surgem naturalmente
- ❖ Perguntas sucessivas justificam memória
- ❖ Serviços externos permitem discutir MCP
- ❖ Especialistas justificam uma arquitetura MAS
- ❖ Qualidade, custo, latência e confiabilidade são mensuráveis

Evolução do estudo de caso

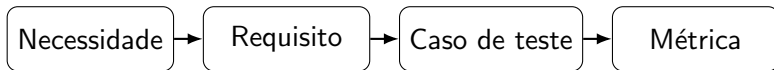
Entregável	Prazo	Versão
1	06/09	Especificação + baseline com LLM
2	13/09	Workflow/ReAct + memória + integração/MCP
3	20/09	MAS com especialização, skills e planejamento
4	04/10	Tratamento de erros + avaliação e comparação arquitetural

O Entregável 4 compara todas as versões: o que for medido hoje será reutilizado até o fim

Antes de programar: especificar

1. Qual problema queremos resolver?
2. Quem utilizará o sistema?
3. Quais entradas e saídas?
4. O que o sistema deve fazer?
5. Quais propriedades de qualidade importam?
6. Quais restrições existem?
7. Como saberemos se o sistema é bom?

Da necessidade ao teste



“O pesquisador não pode perder o prazo” → RF-01 → pergunta de teste → acerto/erro

Um requisito que não chega até a última caixa não poderá ser avaliado no Entregável 4

Exemplo de especificação

Usuário-alvo: pesquisador preparando uma submissão

Entrada: documento da chamada + pergunta

Saída: resposta estruturada com resposta, evidência e confiança

Objetivo: reduzir esforço de leitura e o risco de perder requisitos importantes

Requisitos funcionais (RFs)

- ❖ **RF-01:** identificar prazos
- ❖ **RF-02:** listar documentos obrigatórios
- ❖ **RF-03:** identificar critérios de elegibilidade
- ❖ **RF-04:** responder usando o conteúdo do documento
- ❖ **RF-05:** indicar quando uma informação não foi encontrada
- ❖ **RF-06:** fornecer evidência que sustente a resposta

Requisitos não funcionais (RNFs)

- ❖ **RNF-01:** saída estruturada
- ❖ **RNF-02:** evitar afirmações sem evidência
- ❖ **RNF-03:** tempo de resposta aceitável
- ❖ **RNF-04:** custo controlado
- ❖ **RNF-05:** execução reproduzível para testes
- ❖ **RNF-06:** arquitetura passível de evolução incremental

Requisito vago vs. requisito verificável

Vago

“O sistema deve ser preciso”

“O sistema não deve alucinar”

“O sistema deve ser rápido”

Verificável

RF-01: em 10 editais de teste, extrair a data de submissão correta em pelo menos 8

RF-06: toda afirmação deve vir com um trecho literal do documento; trechos inexistentes contam como erro

RNF-03: latência mediana abaixo de 10 s por pergunta

Teste: consigo escrever, hoje, o código que decide se o requisito foi atendido?

Escopo, não-objetivos e premissas

- ❖ **Escopo:** o que entra nesta versão
- ❖ **Não-objetivos:** o que fica *deliberadamente* de fora
- ❖ **Premissas:** o que assumimos como verdadeiro

Exemplo:

- ❖ escopo: um edital em português, em texto, por vez
- ❖ não-objetivos: OCR de PDF escaneado, múltiplos editais, submissão automática
- ❖ premissas: o documento cabe na janela de contexto

Escrever os não-objetivos protege o cronograma de execução

Critérios de sucesso desde o início

- ❖ Correção e completude
- ❖ Fidelidade da evidência
- ❖ Capacidade de admitir que não sabe
- ❖ Latência
- ❖ Número de chamadas ao modelo
- ❖ Custo aproximado

De critério a medida

Como medir	Quando usar / limitações
Verificação determinística	Há uma resposta de referência (data, valor, lista, formato). Barato e objetivo; frágil a paráfrases
Rubrica humana	Respostas abertas. Confiável, porém caro; exige critério escrito e notas registradas
LLM como juiz	Escala bem, mas precisa ser calibrado contra alguns exemplos rotulados à mão

Meça também a **abstenção**: acertar que a informação não está no documento é um acerto

Conjunto de avaliação congelado

- ❖ Escreva os casos **antes** de ajustar o prompt
- ❖ Congele entradas e respostas de referência do Entregável 1 ao 4
- ❖ Três casos é o mínimo do enunciado: com 3 casos, um acerto vale 33 pontos percentuais
- ❖ Entre 10 e 20 casos já permitem comparar arquiteturas com alguma confiança
- ❖ Cubra: caso normal, caso composto, caso ambíguo, informação ausente e entrada malformada

Se o conjunto mudar, reexecute o baseline: comparação exige a mesma régua

O que registrar em cada execução

- ❖ modelo e versão; temperatura; data da execução
- ❖ versão do prompt
- ❖ saída bruta de cada caso
- ❖ latência por caso
- ❖ tokens de entrada e saída, número de chamadas
- ❖ custo estimado

Um resultado sem essas informações não pode ser comparado com os dos próximos Entregáveis

O que é um baseline?

Definição operacional

O baseline é a solução **mais simples, implementável e avaliável** que fornece um ponto de referência para o problema

Ele não precisa:

- ❖ resolver todo o problema
- ❖ usar agentes
- ❖ usar LangGraph
- ❖ ter a melhor qualidade possível

Ele precisa ser claro o bastante para permitir comparação futura

Baseline nem sempre é uma chamada ao LLM

Dependendo do problema, um baseline pode ser:

- ❖ uma única chamada ao LLM
- ❖ um pipeline linear simples
- ❖ uma regra ou heurística
- ❖ um classificador tradicional
- ❖ busca + LLM, sem agente
- ❖ LLM + uma ferramenta fixa
- ❖ uma versão reduzida de um processo maior

Três formas legítimas de baseline

Completo

Executa a
tarefa principal

A versão simples
já dá conta

Parcial

Executa um
subconjunto
representativo

O problema completo
é grande demais

Proxy

Fornece uma re-
ferência aproximada

Só estabelece um
ponto de comparação

A escolha depende do problema; o requisito central é preservar a comparabilidade

Exemplo: baseline completo

Problema final: responder perguntas sobre um edital

Baseline:

Documento + Pergunta $\rightarrow$ LLM $\rightarrow$ Resposta

Mesmo sendo simples, ele já executa a tarefa principal. As próximas versões podem melhorar confiabilidade e escalabilidade através do uso de ferramentas, memória, planejamento e especialização

Exemplo: baseline parcial

Problema final: revisão bibliográfica automatizada

O sistema final pode precisar:

- ❖ buscar artigos
- ❖ selecionar estudos
- ❖ ler documentos
- ❖ comparar resultados
- ❖ sintetizar evidências

Baseline parcial:

Dado um conjunto fixo de cinco resumos (abstracts), produzir uma síntese

Exemplo: baseline proxy

Problema final: recomendação personalizada complexa

Baseline proxy:

- ❖ ranking por regra simples
- ❖ recomendação por similaridade
- ❖ LLM com prompt único
- ❖ procedimento manual estruturado

O objetivo não é “vencer” o sistema final, mas fornecer uma referência mensurável

Como escolher o baseline?

Pergunte:

1. Qual é a tarefa mínima representativa do problema?
2. Qual solução simples consegue executá-la?
3. Essa solução pode ser testada?
4. Seus resultados podem ser comparados com versões futuras?
5. A simplificação foi explicitamente documentada?

O baseline preserva a comparabilidade

Ao simplificar, tente manter:

- ❖ o mesmo tipo de entrada
- ❖ o mesmo objetivo principal
- ❖ casos de teste reutilizáveis
- ❖ critérios de avaliação compatíveis

Assim, conseguimos comparar:

baseline $\rightarrow$ v2 $\rightarrow$ MAS $\rightarrow$ versão final

Não sabote o seu baseline

Um baseline artificialmente fraco torna a comparação final inútil

- ❖ Use um prompt honesto, que você defenderia
- ❖ Use o mesmo modelo e as mesmas entradas das versões seguintes
- ❖ Mude uma variável por vez entre versões

Se o baseline já resolve bem o problema, isso é um **resultado**, não um fracasso

E se o problema não “precisar” de MAS?

Isso não é necessariamente um problema

Um bom resultado pode ser:

A arquitetura multiagente não trouxe benefício suficiente para justificar custo e complexidade adicionais

O objetivo do projeto é **avaliar decisões arquiteturais**, não apenas adicionar agentes

Por que não começar com um MAS?

A complexidade adicional realmente trará benefício?

Devemos considerar fatores como:

- ❖ qualidade
- ❖ custo
- ❖ latência
- ❖ robustez
- ❖ complexidade de implementação

O baseline nos fornece um “comparandum” para avaliar a viabilidade de se implementar um MAS na resolução do nosso problema

Implementação mínima

```
def analyze_call(question, document):  
    prompt = f"""  
    DOCUMENTO:  
    {document}  
  
    PERGUNTA:  
    {question}  
    """  
    return llm.invoke(prompt)
```

Apenas uma chamada ao modelo, sem workflows, memória ou mesmo MAS

Casos de teste iniciais

1. Qual é o prazo de submissão? (caso normal)
2. Quais documentos são obrigatórios? (lista)
3. Quem pode participar? (interpretação)
4. Qual é o valor máximo de financiamento? (informação ausente)

O quarto caso é o mais informativo: ele testa se o sistema sabe dizer “não sei”

Demonstração: notebook da Aula 2

No notebook veremos:

- ❖ Configuração de chave e modelo (Groq)
- ❖ Saída estruturada com Pydantic
- ❖ O baseline em uma função de poucas linhas
- ❖ Execução instrumentada: latência, tokens e número de chamadas
- ❖ Verificação determinística e seus limites
- ❖ Tabela de resultados que servirá de referência até o Entregável 4

Limitações que abrem o Entregável 2

- ❖ Documento grande → workflow/recuperação
- ❖ Perguntas compostas → decomposição/ReAct
- ❖ Conversa continuada → memória
- ❖ Fontes externas → ferramentas/MCP

Limitações que abrem o Entregável 3

Possíveis responsabilidades especializadas:

- ❖ elegibilidade
- ❖ requisitos e documentos
- ❖ pesquisa externa
- ❖ planejamento
- ❖ revisão/verificação

Pergunta: quando uma responsabilidade merece virar um agente?

- ❖ Responder adequadamente esta questão é essencial numa eventual transição para a arquitetura MAS

Regra prática para o Entregável 1

Cada grupo deve classificar seu baseline como:

- ❖ **baseline completo;**
- ❖ **baseline parcial;** ou
- ❖ **baseline proxy/substituto.**

E justificar:

1. por que essa escolha é adequada
2. o que foi deliberadamente excluído
3. como ela permitirá comparação nas próximas versões

Entregável 1: o que entregar

Especificação + Baseline — prazo: 7 de setembro (segunda)

- ❖ Notebook .ipynb, com saídas salvas
- ❖ Especificação: problema, usuário, escopo e não-objetivos
- ❖ Requisitos funcionais e não funcionais, verificáveis
- ❖ Baseline executável, classificado e justificado
- ❖ Pelo menos três casos de teste, com resultados registrados
- ❖ Análise crítica das limitações.

Enunciado completo e template do notebook no Classroom.

Pergunta que acompanhará o projeto

Como demonstrar, ao final da disciplina (e do curso), que a arquitetura final é melhor do que o baseline?